JAVA ASSIGNMENT

1. Differences between Core Java and Advanced Java

Core Java is the fundamental part of Java programming used for developing general-purpose applications, whereas Advanced Java is used for developing dynamic web and enterprise applications. Advanced Java is built on Core Java concepts and provides additional technologies for distributed computing.

Core Java	Advanced Java
Core Java is used to develop standalone applications.	Advanced Java is used to develop web-based and enterprise applications.
It includes basic concepts such as OOP, Exception Handling, Collections, Multithreading, File Handling, and basic JDBC.	It includes technologies such as Servlets, JSP, JDBC, Hibernate, Spring, Spring Boot, and Web Services.
Applications run on a single system.	Applications run in client-server and distributed environments.
Does not require a web server.	Requires a web server or application server like Apache Tomcat.
Focuses on basic programming structure and logic building.	Focuses on enterprise-level application development and database connectivity.
Used for desktop applications and console programs.	Used for web applications, banking systems, e-commerce, and ERP systems.
Easier to learn and forms the base of Java.	More complex and requires knowledge of Core Java.
Example: Calculator, Notepad application.	Example: Online shopping system, banking portal.

Conclusion:

Core Java provides the basic foundation of Java programming, while Advanced Java extends these concepts to develop powerful, secure, and scalable enterprise applications.

2. Main Important Features of Applets in Java

An Applet is a small Java program that runs inside a web browser or applet viewer. It was mainly used to create interactive web applications before modern web technologies like HTML5 and JavaScript became popular.

Features of Applets:

1. **Client-Side Execution:**
   Applets run on the client machine inside a browser using JVM, reducing server load.
2. **Platform Independent:**
   Applets are written in Java bytecode and can run on any system with JVM support.
3. **Secure Execution (Sandbox Model):**
   Applets run in a restricted environment and cannot access local system files directly.
4. **Lifecycle Methods:**
   Applets work using lifecycle methods like init(), start(), stop(), paint(), and destroy().
5. **Graphical User Interface (GUI):**
   Applets support GUI using AWT and Swing components such as buttons, text fields, and labels.
6. **Event Handling:**
   Applets respond to user actions like mouse clicks and keyboard inputs.
7. **Multimedia Support:**
   They can display images, animations, and audio.
8. **Dynamic Content:**
   Applets can update content dynamically without reloading the web page.

Conclusion:
Applets were useful for interactive web applications but are now obsolete due to security issues and modern web technologies.

3. Key Features of Java Servlets

A Servlet is a server-side Java program used to handle client requests and generate dynamic web content. It runs inside a servlet container such as Apache Tomcat.

Features of Servlets:

1. **Server-Side Technology:**
   Servlets run on the server and process client requests.
2. **Platform Independent:**
   Written in Java, so they can run on any operating system.

3. **High Performance:**
   Uses multithreading, where a single servlet handles multiple requests efficiently.
4. **Robust Lifecycle Management:**
   Uses init(), service(), and destroy() methods managed by the servlet container.
5. **Session Management:**
   Maintains user data using sessions and cookies.
6. **Database Connectivity:**
   Can connect to databases using JDBC for data operations.
7. **Secure:**
   Provides better security as processing is done on the server side.
8. **Scalable:**
   Can handle a large number of users simultaneously.
9. **Integration Support:**
   Easily integrates with JSP, Spring, and Hibernate.

Conclusion:
Servlets are an important technology for building dynamic, secure, and scalable web applications.

4. Differences between Applets and Servlets

Applets	Servlets
Applets run on the client side.	Servlets run on the server side.
Executed inside a web browser.	Executed inside a web server or servlet container.
Used for GUI-based applications.	Used for generating dynamic web pages.
Requires JVM in browser.	Requires servlet container like Tomcat.
Limited access to system resources.	Full access to server resources.
Mainly used for user interaction.	Mainly used for business logic processing.
Less secure in modern systems.	More secure and widely used.
Now mostly obsolete.	Still widely used in enterprise applications.

Conclusion:
Applets are client-side GUI programs, whereas Servlets are server-side components used for web application development.

5. Role of JDBC and ODBC in Database Connectivity

Database connectivity allows applications to interact with databases for storing and retrieving data. JDBC and ODBC are two important technologies used for this purpose.

JDBC (Java Database Connectivity):

JDBC is a Java API used to connect Java applications with databases.

Roles:

- Establishes connection between Java application and database.
- Executes SQL queries like SELECT, INSERT, UPDATE, DELETE.
- Retrieves and updates database data.
- Supports transaction management.
- Works with databases like MySQL, Oracle, PostgreSQL.

Features:

- Platform independent.
- Specifically designed for Java.
- Secure and efficient.

ODBC (Open Database Connectivity):

ODBC is a standard API used to connect applications written in different languages to databases.

Roles:

- Provides a common interface for database access.
- Uses ODBC drivers for communication.
- Supports multiple programming languages.
- Allows database switching without changing application code.

Features:

- Language independent.

- Works on multiple database systems.
- Commonly used in Windows applications.

Conclusion:
JDBC is Java-specific database connectivity, while ODBC is language-independent and used for general database access.

6. Differences between ODBC and JDBC

ODBC and JDBC are both used for database connectivity, but they differ in design, usage, and language support. ODBC is a general-purpose database access standard, while JDBC is specifically designed for Java applications.

ODBC	JDBC
ODBC stands for Open Database Connectivity.	JDBC stands for Java Database Connectivity.
It is language independent.	It is specific to Java programming language.
Uses ODBC drivers for database communication.	Uses JDBC drivers for database communication.
Can connect to multiple types of applications written in different languages.	Can only be used in Java applications.
Requires ODBC driver installation on the system.	Does not require external installation; included in Java API.
Slightly slower due to additional layers.	Faster and more efficient for Java applications.
Mostly used in Windows-based applications.	Used in platform-independent Java applications.

Conclusion:
ODBC is a universal database connectivity standard, while JDBC is optimized for Java applications.

7. Differences between Cookies and Sessions in Web Applications

Cookies and sessions are both used to maintain user information in web applications, but they differ in storage location and security.

Cookies	Sessions
Stored on the client-side (browser).	Stored on the server-side.
Less secure because data is stored in the browser.	More secure because data is stored on the server.
Has limited storage capacity (around 4KB).	Can store large amounts of data.

Cookies	Sessions
Can persist even after browser is closed (if set).	Ends when session expires or user logs out.
Easy to implement.	Slightly complex to manage.
Example: Remembering login preferences.	Example: Maintaining logged-in user session.

Conclusion:

Cookies store data on the client side, while sessions store data securely on the server side.

8. Differences between Spring Boot and Traditional Spring Framework

Spring Boot is an extension of the Spring Framework that simplifies application development by reducing configuration complexity.

Spring Framework	Spring Boot
Requires manual configuration.	Provides auto-configuration.
Needs XML or Java-based configuration setup.	Uses annotation-based configuration.
Requires external server setup.	Comes with embedded servers like Tomcat.
More complex to start a project.	Very easy and quick project setup.
Suitable for experienced developers.	Suitable for rapid application development.
Provides flexibility but requires more coding.	Reduces boilerplate code significantly.

Conclusion:

Spring Boot simplifies Spring Framework by providing auto-configuration and faster development.

9. Differences between Standalone Applications and Web Applications

Standalone Applications	Web Applications
Run on a single computer.	Run on client-server architecture.
Do not require internet connection.	Require internet or network connection.
Installed on a local system.	Accessed through web browsers.
Example: Notepad, Calculator.	Example: Online banking, shopping websites.
Limited accessibility.	Accessible from anywhere.
No server required.	Requires web server and database.
Built using Core Java or similar languages.	Built using technologies like Servlets, JSP, Spring.

Conclusion:
Standalone applications run locally, while web applications run over a network and are accessible remotely.

10. Basic Requirements for Executing a Web Application

A web application follows a client-server architecture. Therefore, both client-side and server-side components are required for proper execution.

1. Client-Side Requirements:

- A web browser such as Chrome, Firefox, or Edge.
- Operating system like Windows, Linux, macOS, or Android.
- Internet connection to communicate with the server.
- Basic hardware resources (CPU, RAM).

2. Server-Side Requirements:

- Web server such as Apache HTTP Server or Nginx.
- Application server or servlet container like Apache Tomcat.
- Database Management System (DBMS) such as MySQL, Oracle, or PostgreSQL.
- Server-side programming language (Java, PHP, etc.).

3. Network Requirements:

- HTTP/HTTPS protocol for communication.
- Stable internet or intranet connection.
- DNS system for domain name resolution.

Conclusion:
A web application requires both client and server environments connected through a network to function properly.

Comparison of Java, C, C++, Python, and JavaScript

Comparative Table

Language Type	High-level, object-oriented	Procedural language	Multi-paradigm (procedural + OOP)	High-level, interpreted	High-level, scripting language
Open Source /	Open source (Oracle +	Open source	Open source	Open source	Open source

Commercial	community)				
Execution	Compiler + JVM (bytecode)	Compiled	Compiled	Interpreted	Interpreted (JIT in engines)
OOP Support	Yes (fully object-oriented)	No	Yes	Yes	Yes
Developer Organization	Oracle	Dennis Ritchie (Bell Labs)	Bjarne Stroustrup	Python Software Foundation	Netscape (initial), now ECMA
Current Version (2026)	Java 23+	C11/C17 standard	C++23	Python 3.13+	ECMAScript 2025
Primary Purpose	Enterprise, web apps	System programming	System + application programming	AI, ML, scripting	Web development
Common Applications	Banking, Android apps	OS development, embedded systems	Games, real-time systems	AI, data science, automation	Front-end web apps
Database Support	Strong (JDBC, ORM)	Limited	Strong	Strong	Strong (Node.js, APIs)
Memory Management	Automatic (GC)	Manual	Manual	Automatic	Automatic
Security Features	High	Low–Medium	Medium	High	Medium
Performance	High	Very high	Very high	Medium	High
Platform Independence	Yes (WORA)	No	No	Yes	Yes
Ease of Learning	Moderate	Difficult	Difficult	Easy	Easy
IDEs	Eclipse, IntelliJ IDEA	Code::Blocks, Dev-C++	Visual Studio	PyCharm, VS Code	VS Code, WebStorm
Compilation Output	Bytecode (.class)	Machine code	Machine code	Bytecode/interpreted	Browser-executed scripts
Advantages	Secure,	Fast, low-	Powerful	Simple,	Essential for

	portable, scalable	level control	OOP, fast	versatile	web UI
Limitations	Slower than C/C++	No OOP support	Complex memory	Slower execution	Browser dependency

Suitability of Languages

- **System Programming: C, C++**
- **Enterprise Applications: Java**
- **Web Development: JavaScript (frontend), Java (backend)**
- **Artificial Intelligence: Python**
- **Mobile Application Development: Java (Android), Python (cross-platform tools)**

Conclusion

Programming languages differ based on their design, performance, and application areas. C is a foundational language used for system programming and provides high performance with low-level memory access. C++ extends C by adding object-oriented features and is widely used in game development and real-time systems. Java is a platform-independent language mainly used for enterprise and Android applications due to its security and scalability. Python is a simple and highly readable language popular in artificial intelligence, machine learning, and automation tasks. JavaScript is the backbone of modern web development, enabling interactive web pages and dynamic user interfaces. Each language has its own strengths and limitations, and the choice of language depends on the specific requirements of the application domain. Overall, understanding multiple languages helps developers choose the right tool for efficient software development.